

```
// layout.js
export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        <Navbar /> // appears once globally
        {children}
      </body>
    </html>
  )
}
```

```
}
```

What is page.js?

This file contains ONLY the content for that page. Your hero section, projects grid, and text are inside page.js.

CHAPTER 2 — IMAGES, GRIDS & MASONRY

Why Your Images Look Huge

Images stretch to the width of their container. If the container is large (full screen width), the image grows with it.

object-cover Explained

This keeps the image shape proportional while cropping the excess.

```

```

Grid vs Masonry

```
GRID (Uniform)
[ Img ][ Img ][ Img ]
[ Img ][ Img ][ Img ]

MASONRY (Pinterest style)
[ Img ]
[ Img ][ Img ]
[ Img ][ Img ][ Img ]
```

Correct Masonry CSS

```
.masonry {
  column-count: 3;
  column-gap: 1.5rem;
}

.masonry-item {
  break-inside: avoid;
  margin-bottom: 1.5rem;
}
```

CHAPTER 3 — REACT BASICS YOU MUST KNOW

What is the `projects[]` array?

This array stores all project data you display. Think of it like rows in a table.

```
const projects = [  
  { title: "Modern Villa", img: "/villa.jpg" },  
  { title: "Bungalow 1", img: "/bungalow.jpg" }  
];
```

What `.map()` Actually Does

It loops and creates UI for each project.

```
projects.map((p) => (  
  <div>  
    <img src={p.img} />  
    <h3>{p.title}</h3>  
  </div>  
));
```

JSX = HTML inside JavaScript

JSX allows us to write UI with HTML-like syntax, but inside JavaScript code.

CHAPTER 4 — FRONTEND DESIGN ELEMENTS

Hero Sections Explained

[illegible]

absolute vs relative

absolute positions an element anywhere on screen. The parent **MUST** be relative.

```
<section className="relative">
  <img />
  <div className="absolute inset-0">Overlay</div>
</section>
```

Tailwind Responsive Syntax

```
text-lg           // mobile
sm:text-xl       // small screen+
md:text-2xl      // medium screen+
lg:text-4xl      // large screen+
```

CHAPTER 5 — BACKEND (SPRING + MONGO)

Model / Document

In MongoDB, each record is called a document. In Spring Boot, a Java class represents this document.

```
@Document("bookings")
public class Booking {
    private String name;
    private Long phone;
}
```

Repository

The interface that gives CRUD operations automatically.

```
public interface BookingRepo extends MongoRepository<Booking, String> {}
```

Controller

```
@PostMapping("/booking")
public Booking createBooking(@RequestBody Booking b) {
    return repo.save(b);
}
```

CHAPTER 6 — SYSTEM ARCHITECTURE FLOW

Full System Architecture

USER → React Frontend
↓ API request (JSON)
SPRING BOOT BACKEND
↓ CRUD operations
MONGO DB DATABASE
↓ response
FRONTEND UI (updates)